Project Title: RiazAI – Intelligent Audio Practice Analysis System

Student Name: Devansh
Roll Number: 23EG106A12
Department: Artificial Intelligence / Computer Science
University / College: Anurag University
Guide Name: K Vikram
Academic Year: 2025–2026
Project Repository (GitHub Link): https://github.com/Violindeva09/RiazAI

Abstract

RiazAI is a lightweight backend system designed to analyse short audio practice recordings and generate measurable performance insights. The objective of the project is to demonstrate how raw audio data can be converted into structured metrics using backend processing techniques. The system is developed using the Spring Boot framework and follows a layered MVC architecture consisting of controller, service, and model components.

The application allows users to upload an audio file representing a short practice session. The backend processes the audio data and performs statistical analysis on the raw byte stream extracted from the file. Key features such as average amplitude and signal variance are computed to evaluate the quality and stability of the recording. These values are then used to derive performance metrics, including accuracy, stability, and consistency.

Based on these metrics, the system generates rule-based feedback that provides simple coaching suggestions to the user. Additionally, the system produces trend data to visualise performance improvement. The project demonstrates how backend systems can process audio data and provide intelligent insights for music practice analytics.

---

1. Introduction

Music practice often relies on subjective feedback from instructors or self-evaluation by performers. However, with advancements in software systems, it is possible to analyze audio recordings programmatically and extract meaningful insights about performance quality.

RiazAI is designed to demonstrate how audio recordings can be processed using backend technologies to generate performance metrics. The system evaluates uploaded audio files and computes indicators that reflect the stability and consistency of the sound signal.

The project focuses on building a prototype backend service capable of analysing audio recordings and generating simple feedback. This concept can be extended to develop intelligent music practice platforms that help users track and improve their performance.

---

2. System Architecture

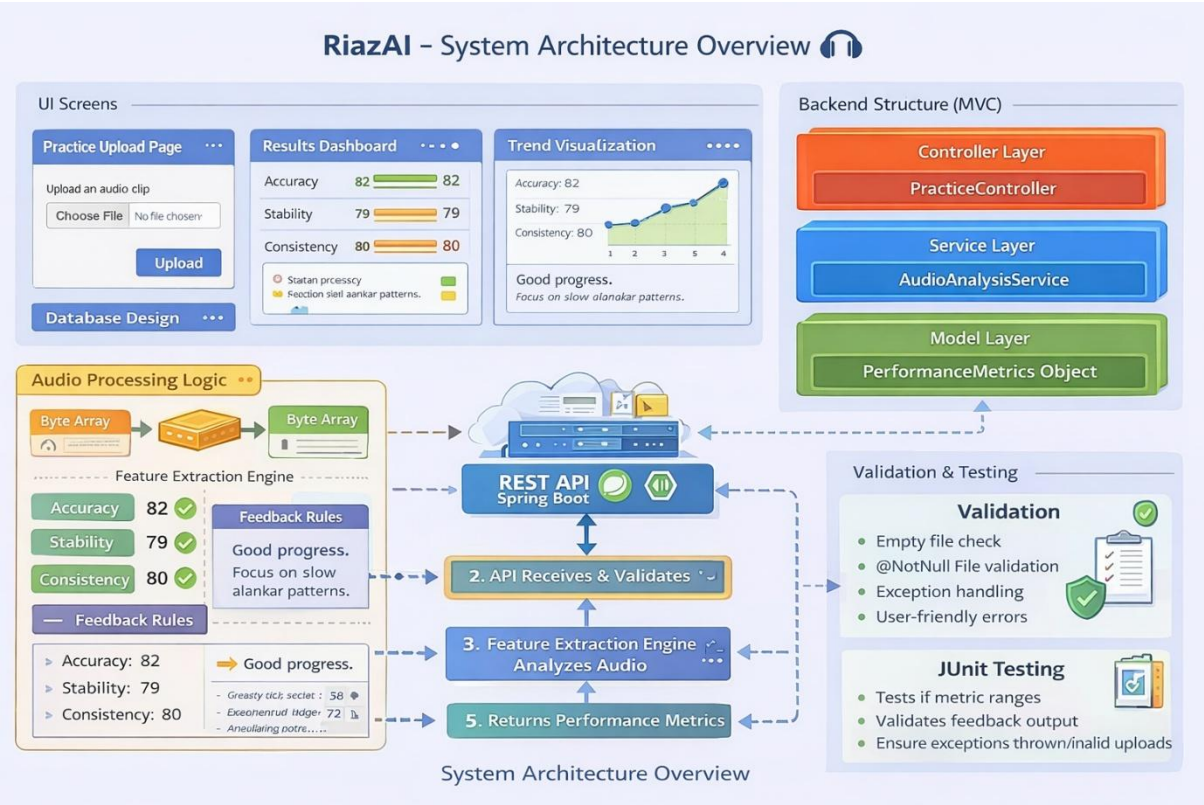


Figure 1: System Architecture of RiazAI

The system follows a layered MVC architecture using the Spring Boot framework. This architecture divides the application into multiple layers to improve modularity and maintainability.

The Controller Layer handles incoming HTTP requests and manages communication between the user interface and the backend service. The controller validates the uploaded audio file and forwards it to the service layer for processing.

The Service Layer contains the core logic of the application. The audio analysis service reads the uploaded file, converts it into a byte array, and extracts statistical features from the audio data.

The Model Layer stores the computed performance metrics and feedback results. These values are returned to the user interface for visualization.

This architecture ensures separation of concerns and allows the system to be easily extended in the future.

3. System Workflow

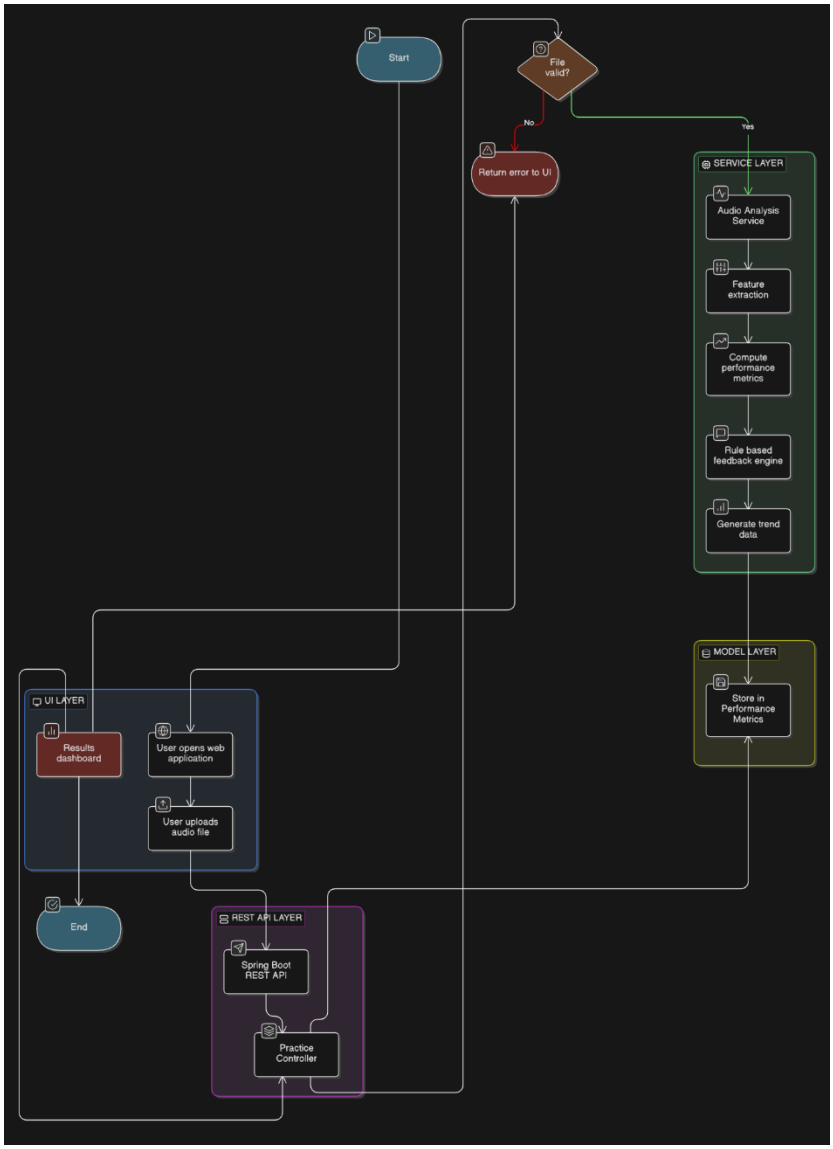
Figure 2: Flowchart of RiazAI

The system workflow begins when the user uploads an audio recording through the web interface. The backend controller receives the file and performs input validation to ensure that the file is not empty.

Once validated, the audio file is forwarded to the service layer, where it is converted into a byte array. The system then calculates statistical features such as average amplitude and variance from the audio data.

These features are used to compute performance metrics including accuracy, stability, and consistency. A rule-based feedback module generates suggestions based on these scores.

Finally, the computed results are returned to the frontend and displayed to the user.



4. Database Design

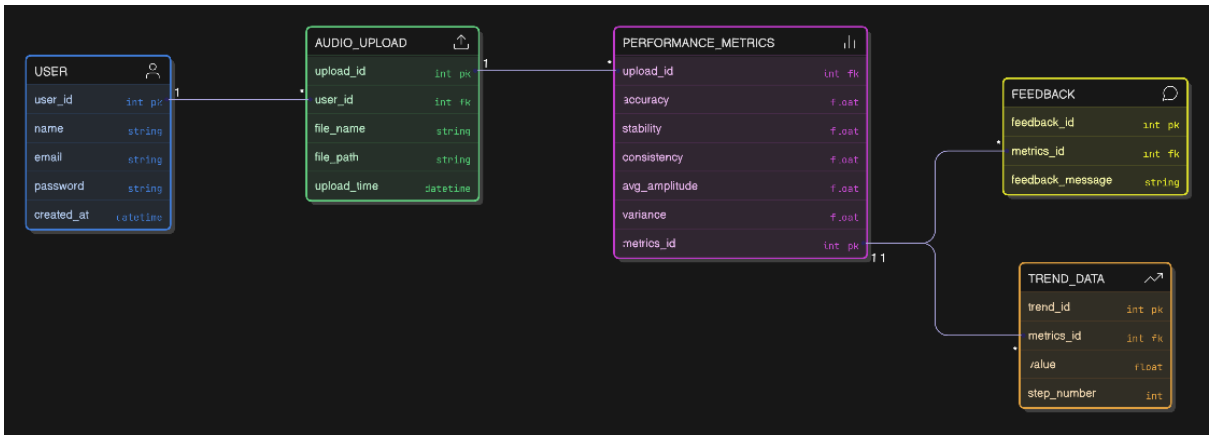


Figure 3: ER Diagram of RiazAI

Although the current implementation processes audio data in memory without persistent storage, a logical database structure can be defined for future scalability.

The proposed ER diagram includes entities such as User, Audio Upload, Performance Metrics, Feedback, and Trend Data.

The User entity stores information related to the system user, while the Audio Upload entity stores metadata about uploaded practice recordings. The Performance Metrics entity contains calculated values derived from the audio analysis. Feedback messages and trend data are associated with these metrics to represent performance insights.

This structure allows the system to be extended with database support in future versions.

---

5. Implementation

The backend of the system is implemented using Spring Boot with a layered MVC design pattern.

The controller handles HTTP requests and validates uploaded files. The service layer processes the audio data and performs statistical calculations on the byte stream extracted from the audio file.

Two primary features are extracted from the audio signal:

Average Amplitude – Represents the overall loudness of the recording.

Variance – Represents fluctuations in the sound signal.

These values are used to derive the following performance metrics:

Accuracy
Stability
Consistency

The system also generates rule-based feedback messages to guide the user toward improving their practice performance.

---

6. Testing

The system was tested using JUnit unit testing to verify the correctness of the audio analysis logic.

Test cases were created to ensure that calculated metrics remain within valid ranges and that feedback messages are generated correctly. The system was also tested using approximately 50–100 sample audio uploads to validate its behaviour under normal operating conditions.

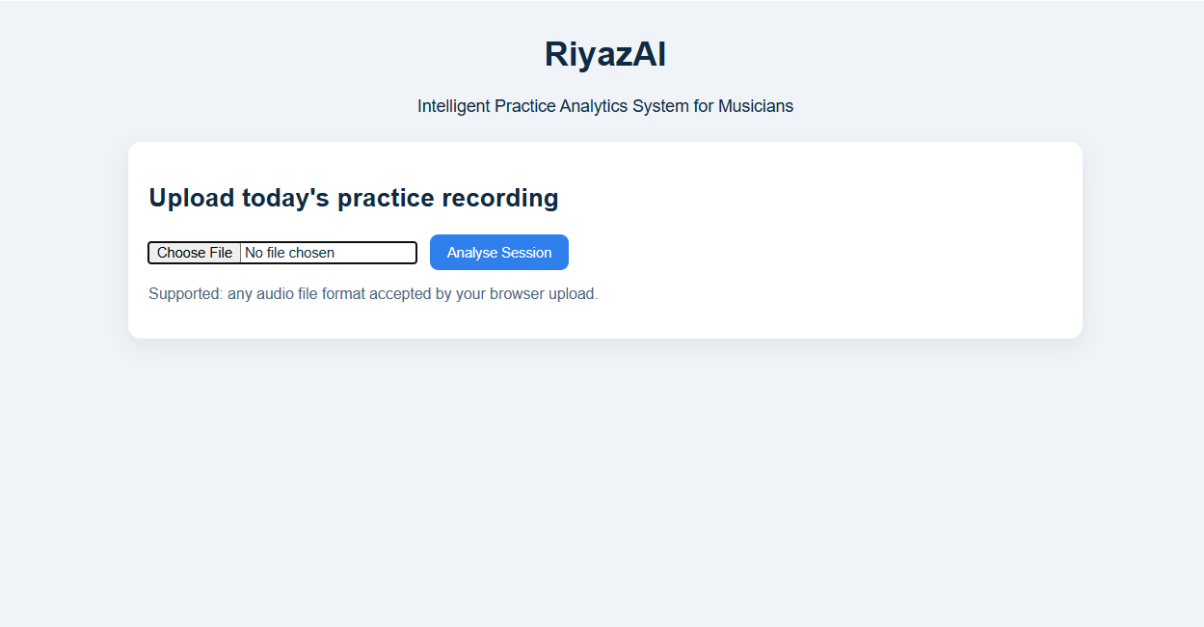
---

7. Results

The output of the system includes calculated performance metrics and feedback messages based on the analysed audio recording.

The system displays values for accuracy, stability, and consistency, along with a suggestion that helps the user understand how to improve their performance.

Additionally, a trend curve is generated to visualise simulated performance improvement over time.



RiyazAI

Intelligent Practice Analytics System for Musicians

Upload today's practice recording

Choose File Arabhi geetam.mp3 Analyse Session

Supported: any audio file format accepted by your browser upload.

RiyazAI

Intelligent Practice Analytics System for Musicians

Upload today's practice recording

Choose File No file chosen Analyse Session

Supported: any audio file format accepted by your browser upload.

Session Report Arabhi geetam.mp3

Accuracy	Note Stability	Consistency
79.42%	95.88%	86.83%

Improvement trend (last 6 sessions)

1. Session 1: **74.83%**
2. Session 2: **77.23%**
3. Session 3: **79.63%**
4. Session 4: **82.03%**
5. Session 5: **84.43%**
6. Session 6: **86.83%**

Personalised feedback

Excellent session. Your pitch control and note steadiness are strong—keep this routine and increase tempo gradually.

---

8. Future Scope

The current implementation uses simple statistical analysis for evaluating audio recordings. Future improvements may include integrating advanced digital signal processing techniques such as Fast Fourier Transform (FFT), pitch detection, and tempo analysis.

Machine learning algorithms can also be introduced to provide more accurate performance evaluation. Furthermore, the system can be integrated into larger music-technology platforms such as VirtuoNet, enabling collaborative music practice analytics.

---

9. Conclusion

RiazAI demonstrates how backend systems can analyze audio recordings and generate structured performance insights. The project highlights the potential of combining audio processing with backend technologies to build intelligent music practice tools.

The modular architecture of the system allows future expansion with advanced audio analysis algorithms and machine learning techniques, making it a strong foundation for intelligent music analytics platforms.